

Department of Electrical and Information Engineering



Threat Containment Strategies for Compromised xApps in Open RAN

Undergraduate Project Presentation

Supervisor: Dr. Chatura Seneviratne
Co-Supervisor: Prof. Dr. An Braeken
Co-Supervisor: Mr. Pramitha Fernando

Dewmith M.K.J. - EG/2021/4474
Dilshan M.D.K. - EG/2021/4485
Sewinda L.L.D. - EG/2021/4807
Thilohith T.K. - EG/2021/4832

04/09/2026



01 Introduction

From Closed RAN to Programmable Open RAN

- Evolution of the RAN: Transition from **monolithic, vendor-locked proprietary hardware** to **open, disaggregated, and software-based components**.
- Architectural Disaggregation: Breaking the 5G base station (gNB) into logical units: **O-CU** (O-RAN Central Unit), **O-DU** (O-RAN Distributed Unit) and **O-RU** (O-RAN Radio Unit).
- Open Interfaces: Integration of multi-vendor components facilitated by **standardized interfaces like E2, A1, O1 and O2**.

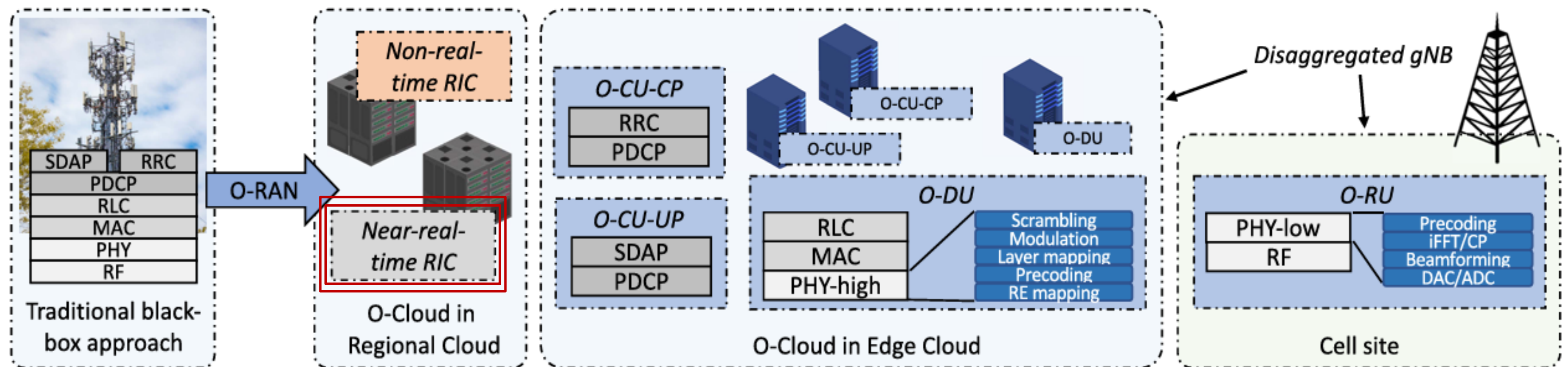
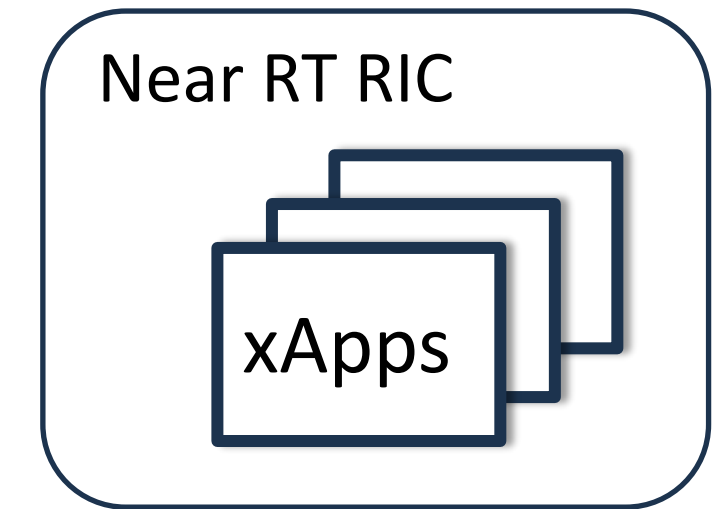


Figure 01: Evolution of traditional base station to a virtualized gNB with disaggregation

Problem Statement : Runtime Trust Gap

Why Is a Compromised xApp a Serious Threat?

- Operate inside the **Near-RT RIC** control plane and consume **sensitive RAN telemetry**
- Can influence **mobility, radio resources** and **QoS**
- May already **possess** valid identity, **permissions** and connectivity
- **Admission time trust** $\neq$ **Runtime trust**



Three Questions Admission Policy Alone Cannot Answer

1) Still Trustworthy?

Is the running xApp still within its expected security and resource profile?

2) Enough Evidence

Which runtime evidence is strong enough to change the xApp trust state?

3) Verified Containment

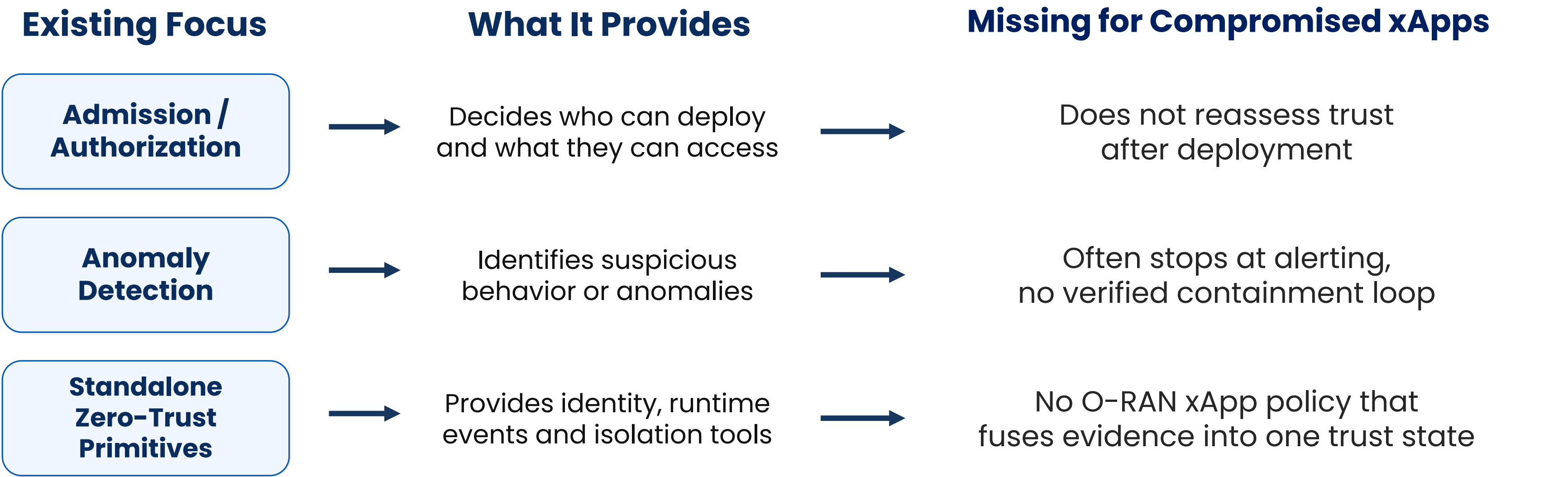
Did enforcement make the xApp actually contained and network isolated?

Required Runtime Trust Loop



Related Work and Research Gap

Existing O-RAN and cloud-native security mechanisms are useful, but they usually cover only one part of the xApp lifecycle.



Secure Onboarding + Continuous Runtime Monitoring + Trust Assessment + Secure Containment

Objectives

Primary Objective: Implement a security framework that enhances the resilience of Near-RT RIC against threats from **compromised xApps**.

This is achieved through the following specific objectives:

1. **Admission Trust:** We use **Secure Onboarding** to verify the **authenticity** of third party xApp deployments ✓
2. **Workload Identity Attestation:** We implement a mechanism for identity and integrity verification of the xApps themselves. ✓
3. **Runtime Behavioral and Resource Abuse Threats:** **Statistical model** approach and **kernel level inspection**. ✓
4. **Automatic Isolation:** Continuous monitoring mechanism for xApps and an isolation pipeline that instantly contains any xApp that violates security protocols. ✓

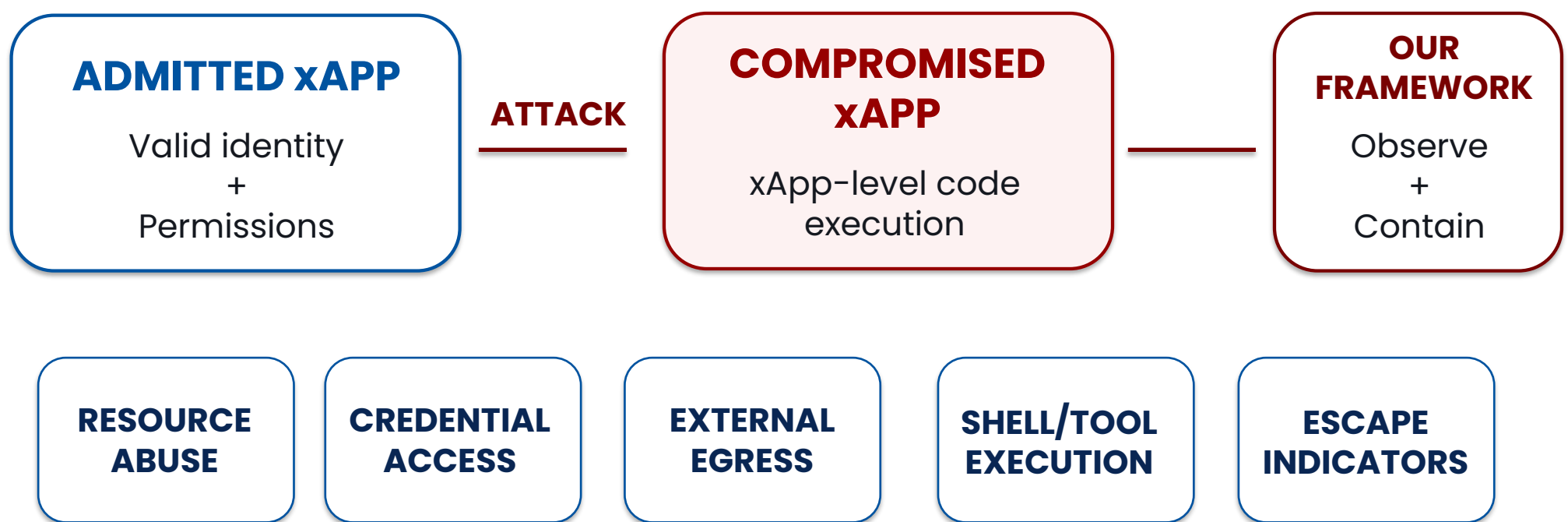


02 Background & Proposed Solution

Threat Model & Scope

Admitted xApp becomes compromised; platform security components remain trusted.

NEAR-RT RIC RUNTIME



TRUSTED BASE: Kubernetes • SPIRE • Monitoring Stack • Policy Engine
• Containment Controller

IN SCOPE

System-level exploitation from one xApp
Resource-pressure anomalies
Runtime identity / credential misuse
Network reachability abuse inside the RIC

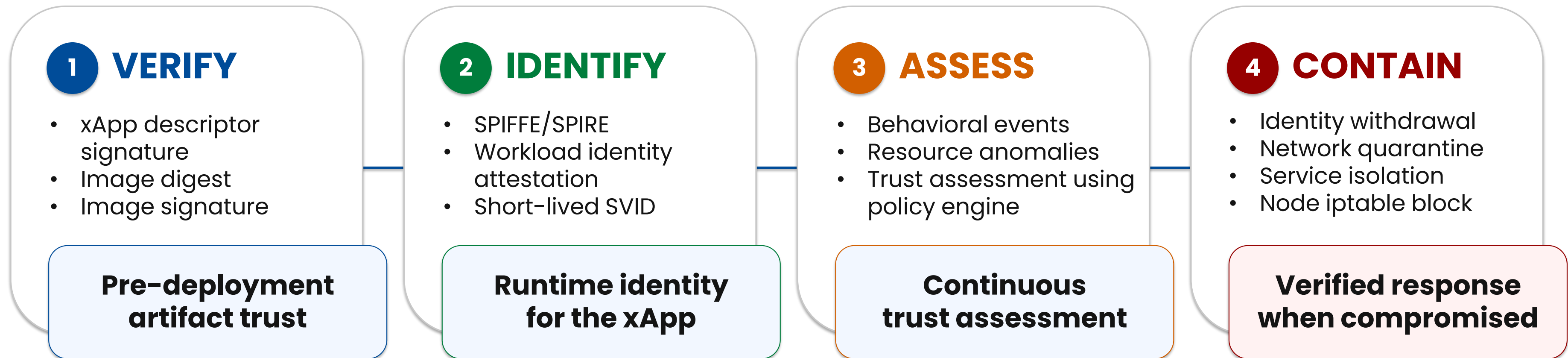
OUT OF SCOPE

Compromised Kubernetes / SPIRE / host
Malicious but well-formed RAN-control logic
E2 data poisoning or physical-layer attacks

Our framework targets runtime compromise evidence at the system and resource layers, not every possible RAN-security attack

Proposed Zero-Trust Framework

Our proposed framework protects an xApp across its lifecycle – before deployment, during execution and after compromise detection.

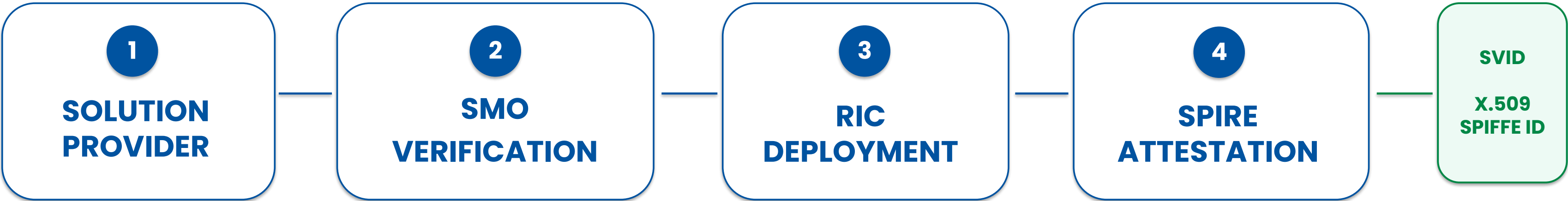


Core idea: trust is not permanent – it is continuously obtained, reassessed and withdrawn when compromise is confirmed.

Secure Onboarding & Workload Attestation

Before runtime trust decisions, the xApp artifact and the running workload are bound to a verifiable identity.

ONBOARDING PIPELINE



Short-lived SVID renews only while the workload remains eligible; registration withdrawal blocks future renewal

WHY THIS MATTERS

- 1

BLOCKS TAMPERED ARTIFACTS
Unsigned or altered packages stop before chart generation.
- 2

BINDS WORKLOAD TO IDENTITY
The running pod is mapped to a SPIFFE workload identity.
- 3

ENABLES IDENTITY CONTAINMENT
On compromise, renewal can be withdrawn as one containment lock.

Makes xApp trust cryptographically earned before runtime and revocable after compromise

Secure Onboarding & Workload Attestation

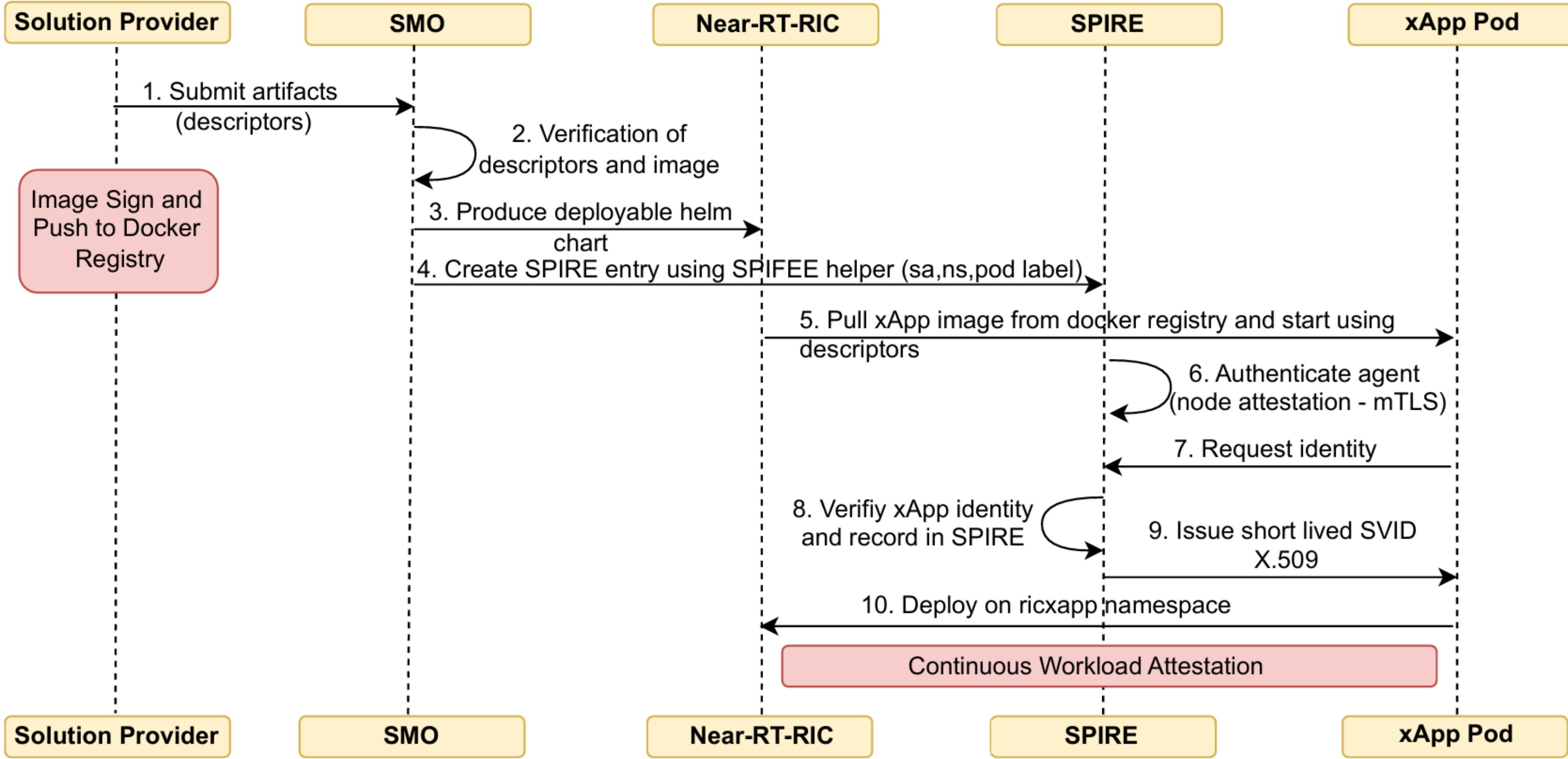


Figure 02: Secure Onboarding Architecture



03

Runtime System Architecture

System Architecture: Threat Detection

After secure onboarding, ZT-XGuard continuously reassesses each running xApp using live runtime evidence.

Two complementary evidence channels

1 Behavioral Channel

- Falco + eBPF kernel events
- Critical policy violation → COMPROMISED

2 Statistical Channel

- cAdvisor + Prometheus resource metrics
- MEWMA / T2 pressure score + dual gate
- Persistence + CPU-corroborated evidence

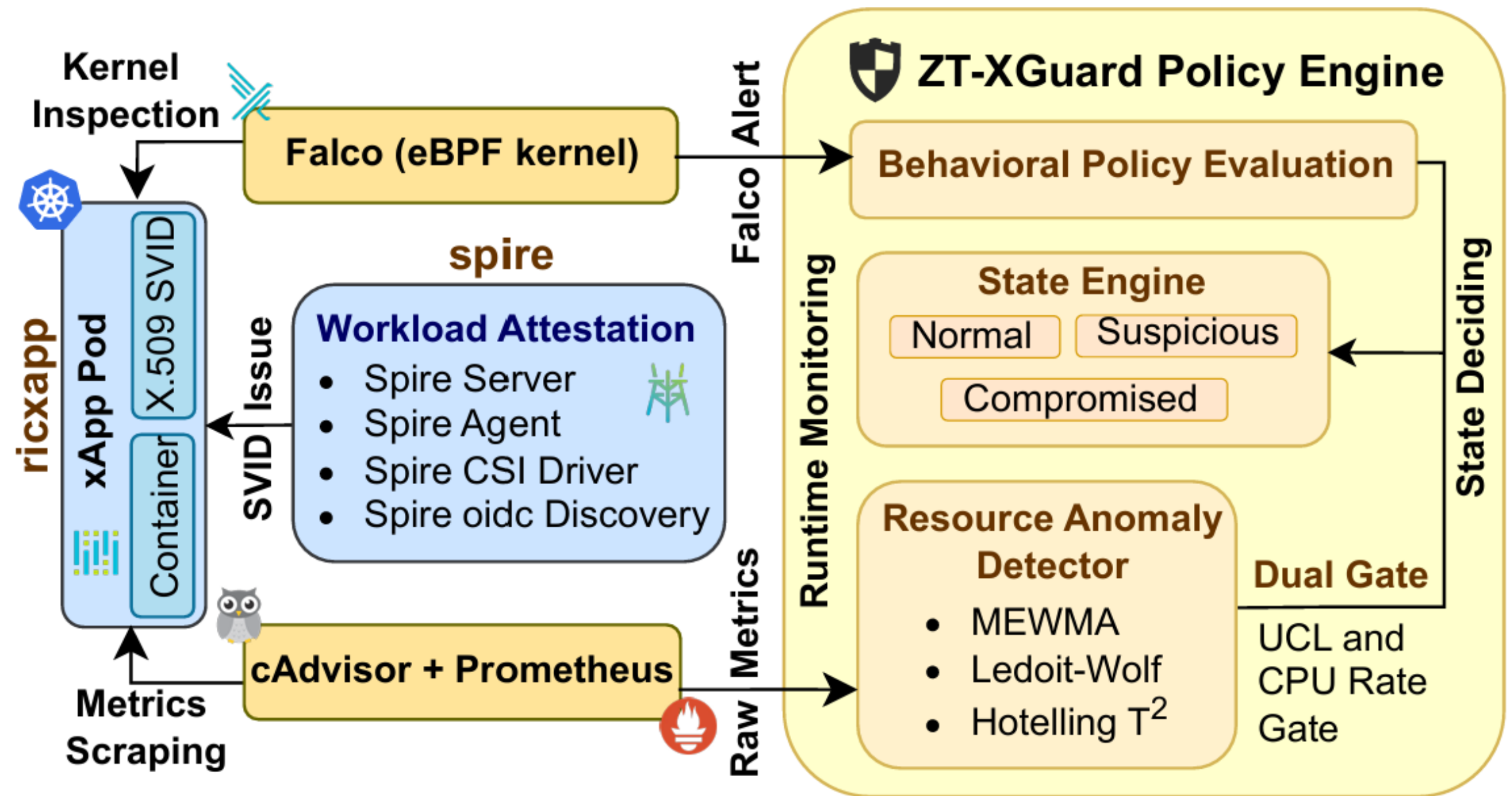


Figure 03: Runtime Detection Subsystem

Kernel behavior and resource telemetry independently propose trust states to the policy engine

Methodology – Statistical Approach

- MEWMA over a simple moving average
- Exponentially-decaying memory of recent behavior
- Multivariate: a single scalar (T^2) fuses CPU, memory-growth, RX and TX

$$Z_t = \lambda \cdot x_t + (1 - \lambda) \cdot Z_{t-1} \text{ ————— Equation 01}$$

$$T^2 = \mathbf{Z}_t^T \Sigma_{lw}^{-1} \mathbf{Z}_t \text{ ————— Equation 02}$$

11 metrics are collected and 4 feed the score

Table 01: Metrics Feed to Model

ID	Units	Meaning
M1	millicores	CPU usage
M4	Bytes/s	Memory growth rate
M5	Bytes/s	Network RX rate
M6	Bytes/s	Network TX rate

Methodology – Adversaries Mapped to MITRE ATT&CK

Table 02: Behavioral Based Attack Mapped to MITRE ATT&CK Techniques

Evaluated Attack	MITRE ATT&CK Technique
Shell Spawn	T1059 - Command and Scripting Interpreter
Sensitive-file access	T1552 - Unsecured Credentials
ServiceAccount-token access	T1528 - Steal Application Access Token
SVID / SPIFFE material access	T1552 - Unsecured Credentials
Malicious tool execution	T1105 - Ingress Tool Transfer
External egress	T1041 - Exfiltration Over C2 Channel
Configuration tamper	T1565 - Data Manipulation
Container-escape attempt	T1611 - Escape to Host
*Resource exhaustion (statistical channel)	T1499 - Endpoint Denial of Service

Source: MITRE ATT&CK® Enterprise — *Techniques & Tactics* <https://attack.mitre.org/>

Policy Engine Trust Assessment

Both detection channels feed one per-xApp trust state, and that state determines the system response.

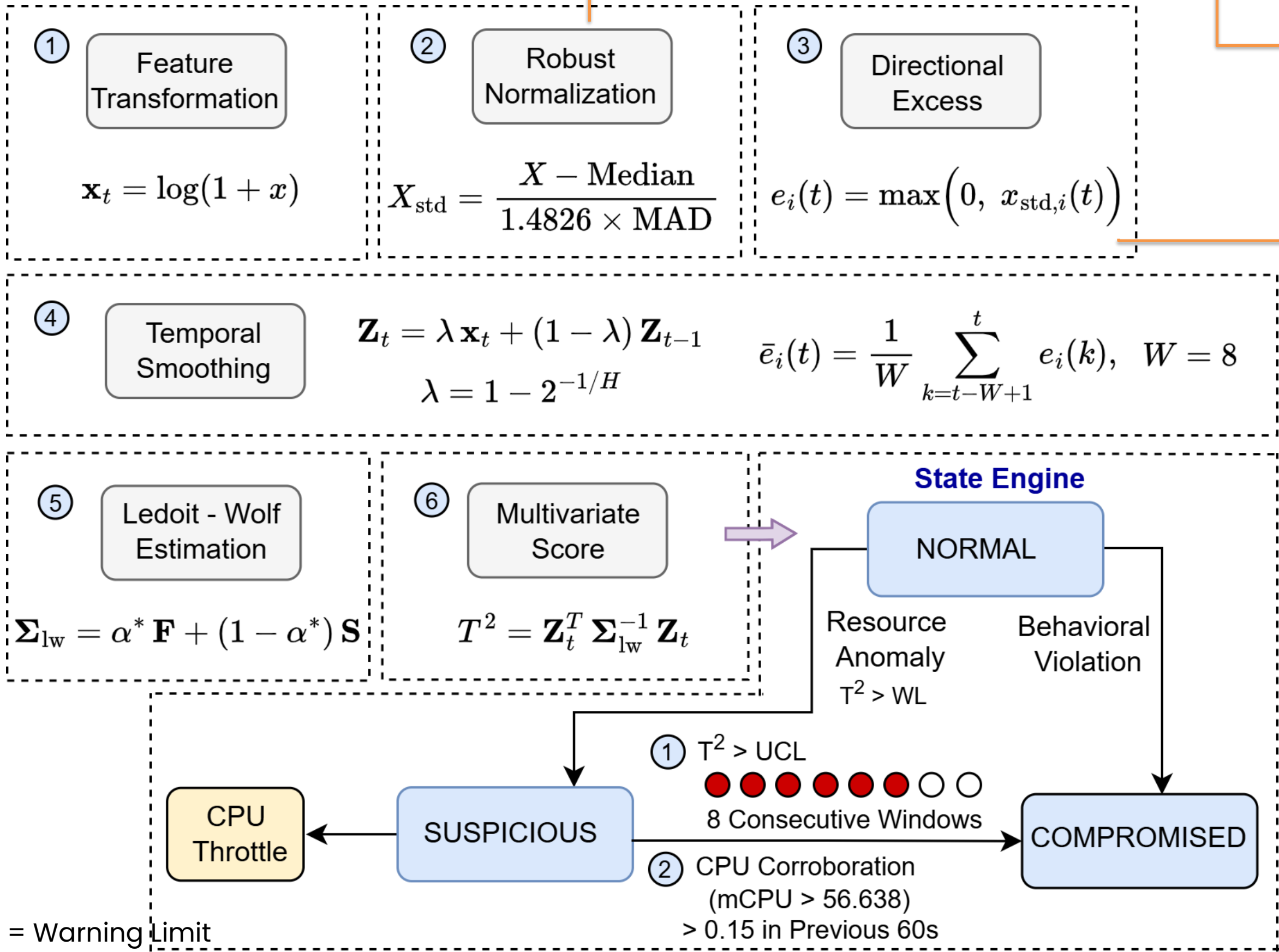


Figure 04: Policy Engine Architecture

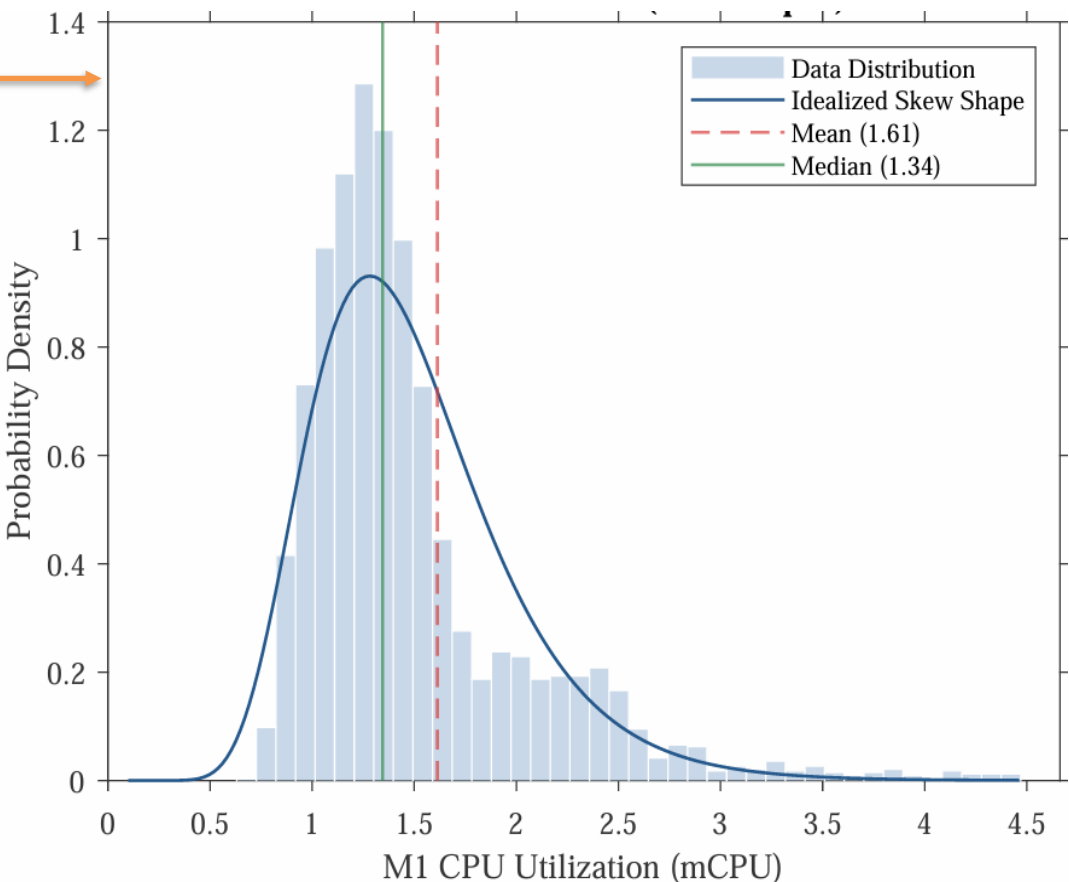


Figure 05: Distribution of the M1 CPU-utilization Metric

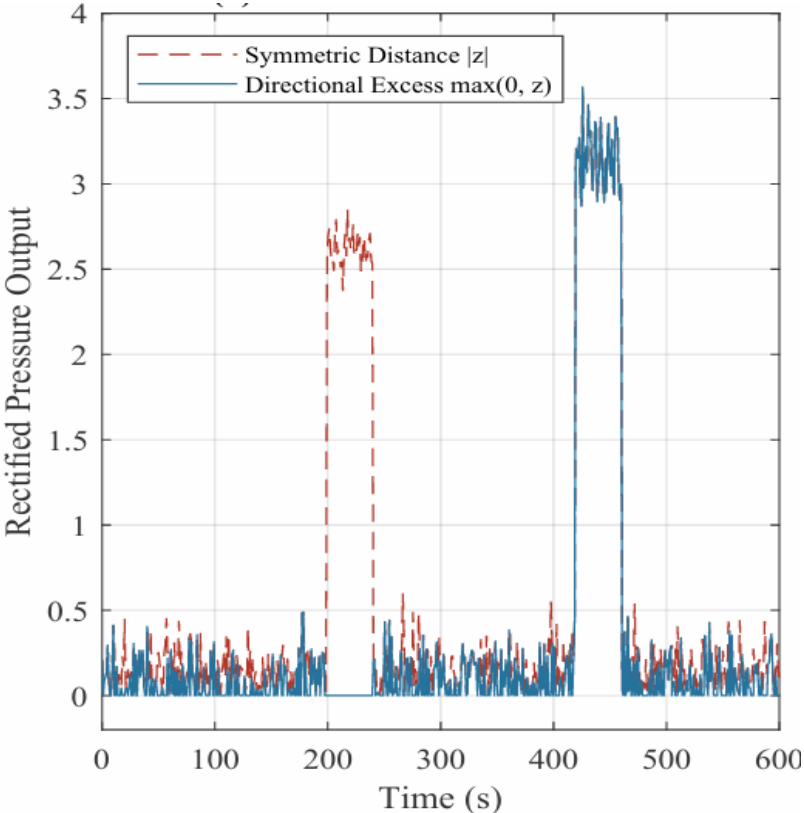


Figure 06: Raw Standardized vs Directional Excess

System Architecture: Threat Containment

ZT-XGuard uses a layered containment orchestrator so the compromised xApp is isolated through multiple complementary actions.

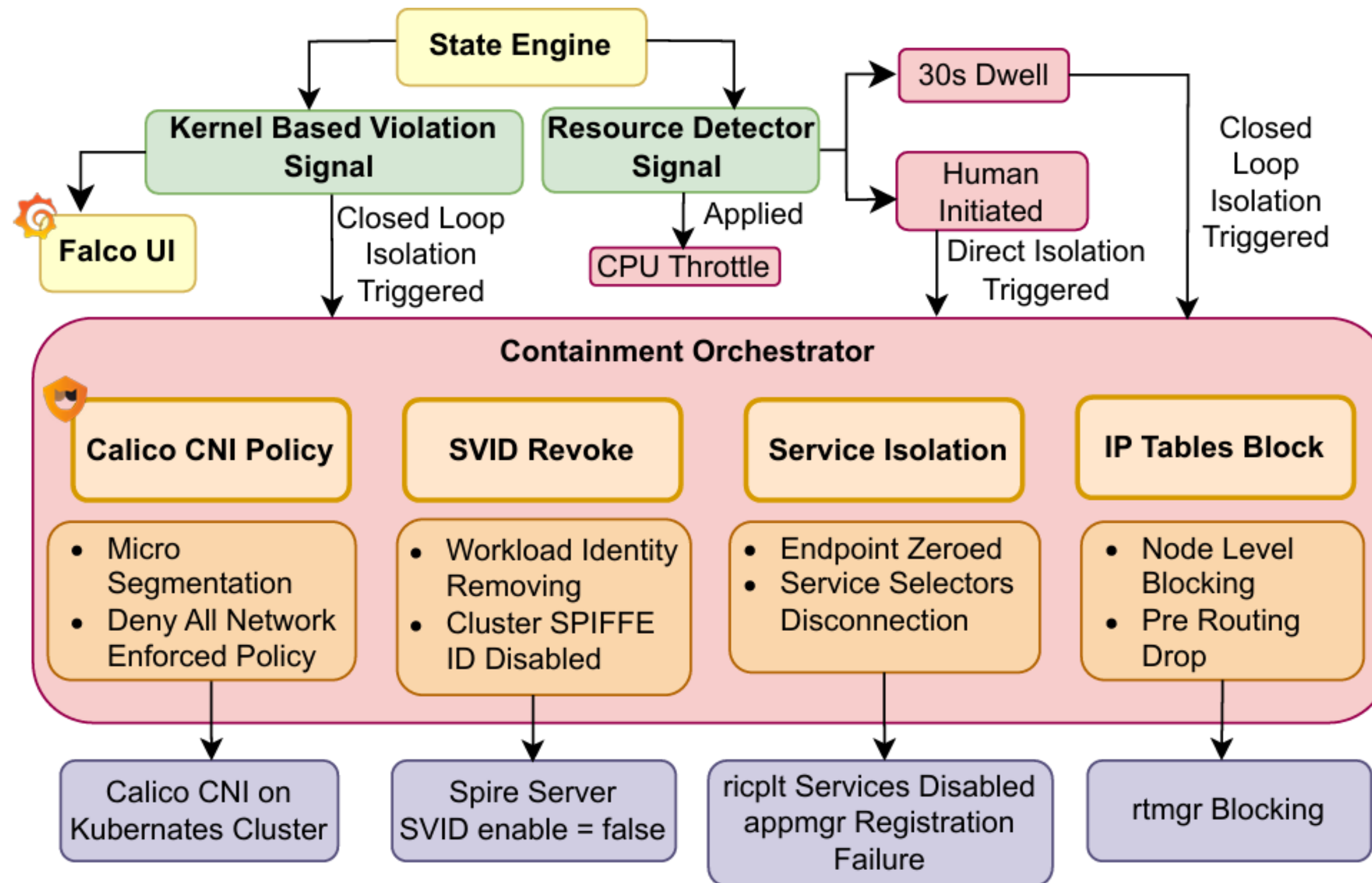


Figure 07: Containment does not rely on a single lock, it combines network, identity, service path and node-level enforcement.



04 Results

Results: Threat Detection and Containment Performance

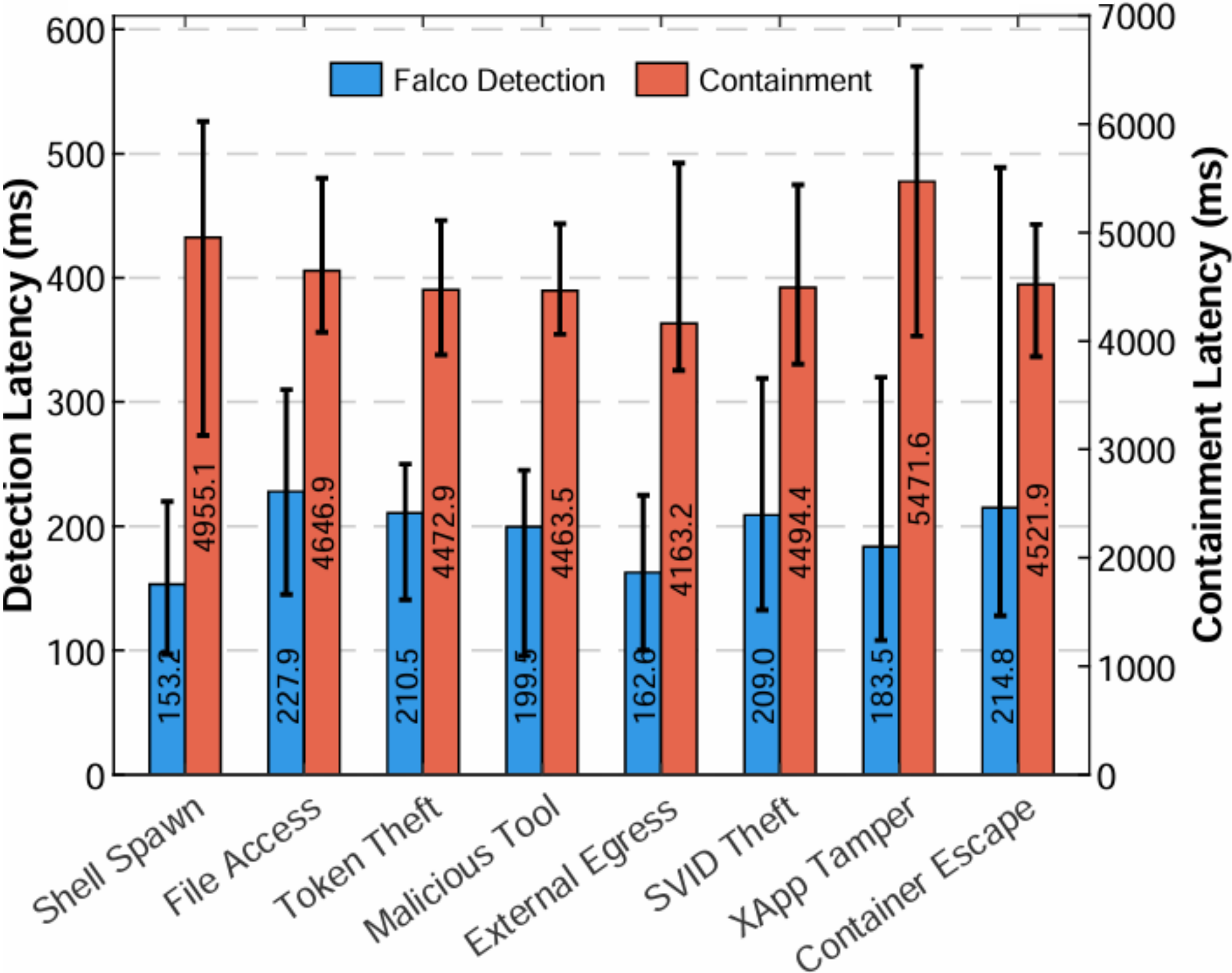


Figure 08: Detection and containment latency for behavioral channel

153–228 ms
Detection latency

4.1–5.5 s
Containment latency

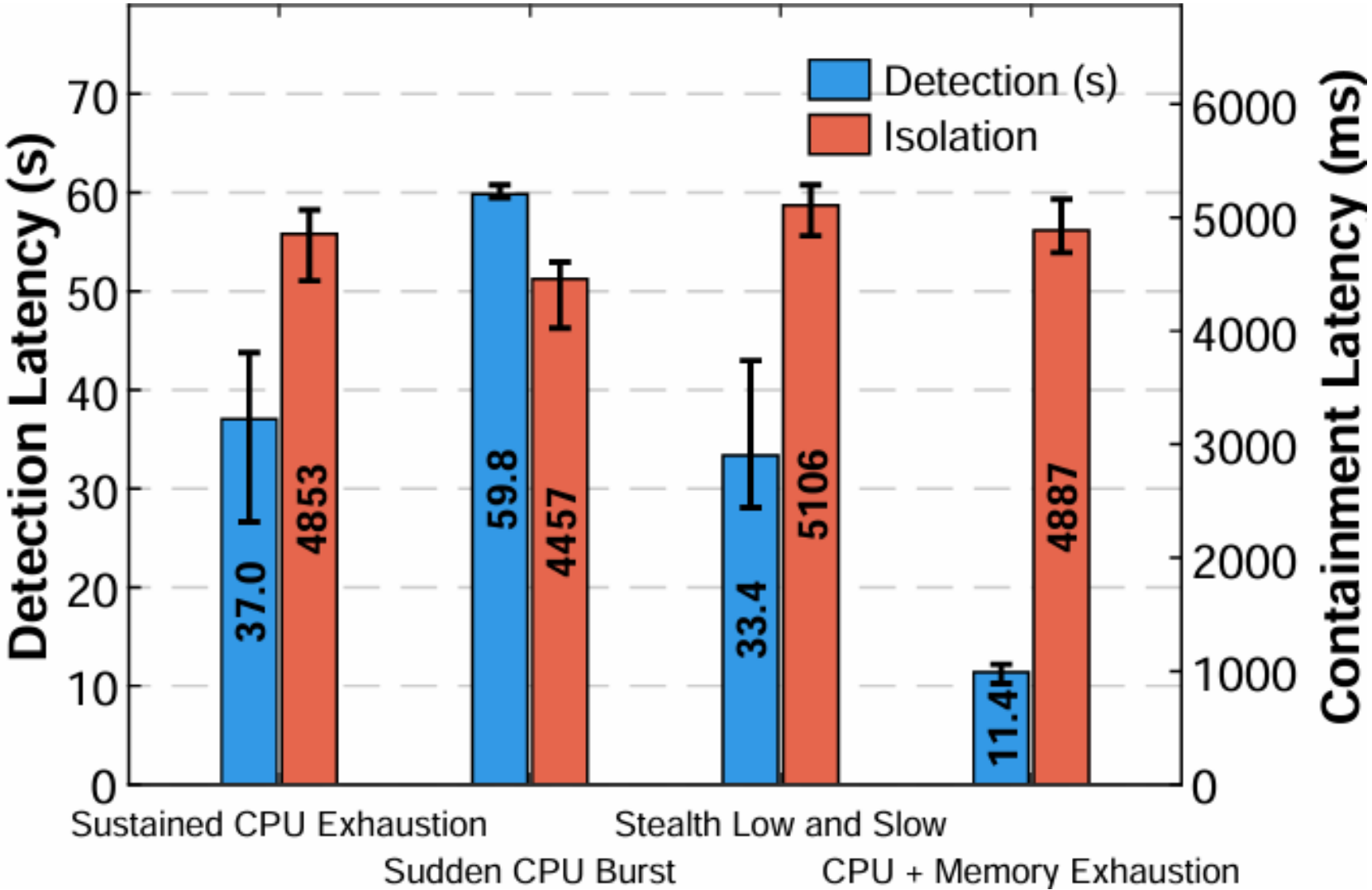
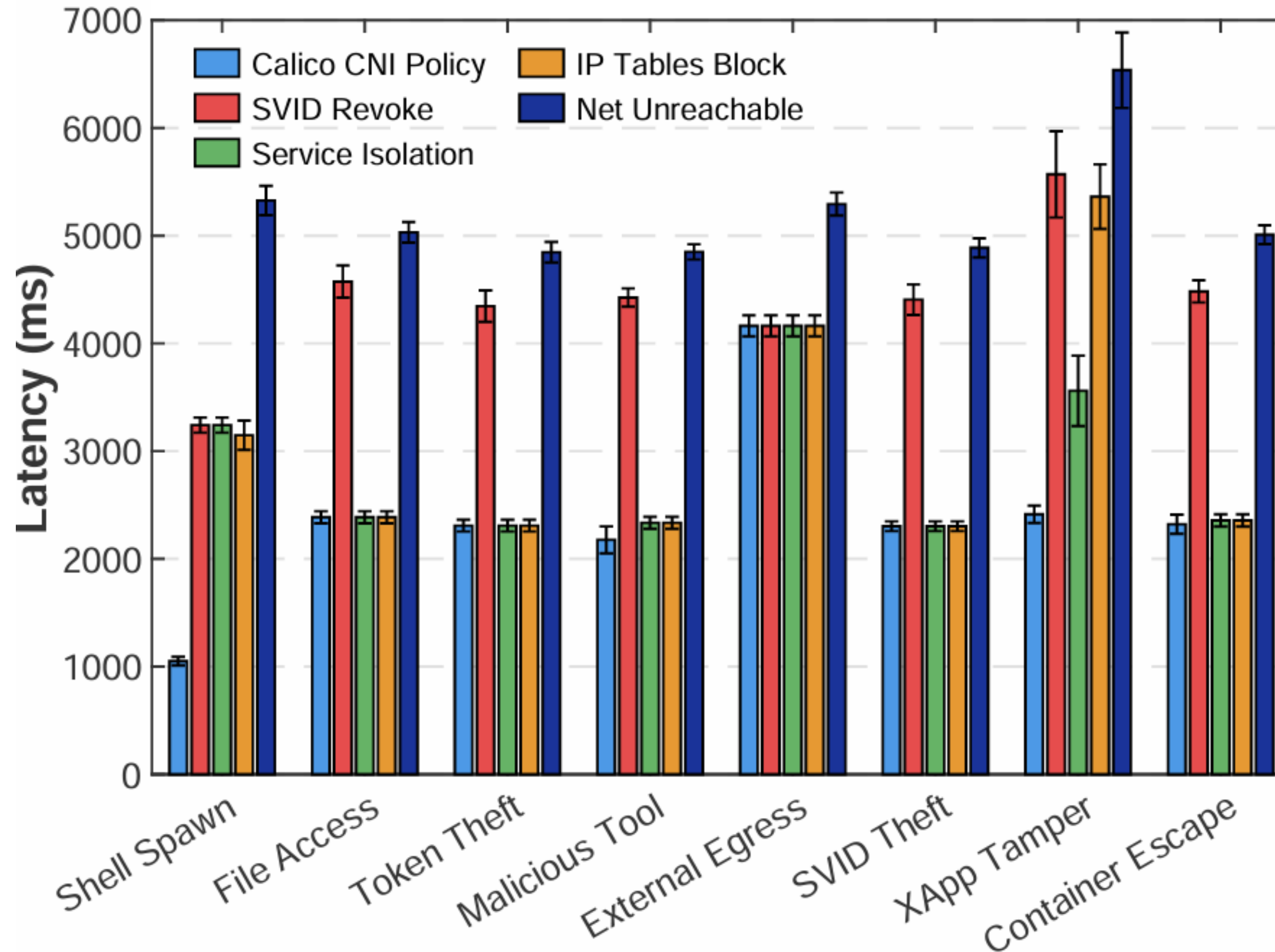


Figure 09: Detection and containment latency for statistical channel

11–60 s
Detection latency

4.5–5.1 s
Containment latency

Results: Per-Mechanism Containment Latency



- 1 Primary locks land first**
Calico NetworkPolicy, Service isolation and node-level block complete in roughly 2.4–3.0 s.
- 2 SPIRE Identity lock is slower**
SPIFFE/SPIRE registration withdrawal completes later, around 4.4 s.
- 3 Verification closes the loop**
The active probe records network and service path unreachability at about 5 s.

Figure 10: Mean per-mechanism containment and verification latency

Are These Latencies Acceptable?

O-RAN specifications do not define a security-response deadline. WG11 treats response time as a metric to measure, not a value to meet. Therefore, we benchmark against the cloud-native standards governing the platform (Kubernetes) the Near-RT RIC actually runs on.

1 BEHAVIORAL CHANNEL

153–228 ms

Sysdig 5/5/5 → detect budget 5 s

CrowdStrike 1-10-60 → detect 60 s

eBPF sees the syscall directly,
and a CRITICAL match needs
no aggregation.

2 STATISTICAL CHANNEL

11–60 s

Sysdig 5/5/5 → triage 5 minutes

No false alarms on clean traffic

We require the anomaly to persist
and be confirmed before isolating a
live mission-critical xApp.

3 CONTAINMENT LATENCY

4.1–5.5 s

Sysdig 5/5/5 → respond 5 minutes

CrowdStrike 1-10-60 → contain 60 minutes

Benchmarks stop at response initiated.
We stop at a failed probe from an
uninvolved pod.

- The Near-RT loop closes in 10 ms – 1 s, so ZT-XGuard operates *outside* it by design
- Previously, a compromised xApp runs unbounded, here **46–94 s to verified isolation**
- Inside 8–16% of the ~10 min an attacker needs.
- Containment limits the exposure window; it does not eliminate it.

Sources:

<https://sysdig.com/555>, <https://www.crowdstrike.com/en-us/cybersecurity-101/cyberattacks/lateral-movement/>,
https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=50951

Verified Removal from the RIC Control Loop

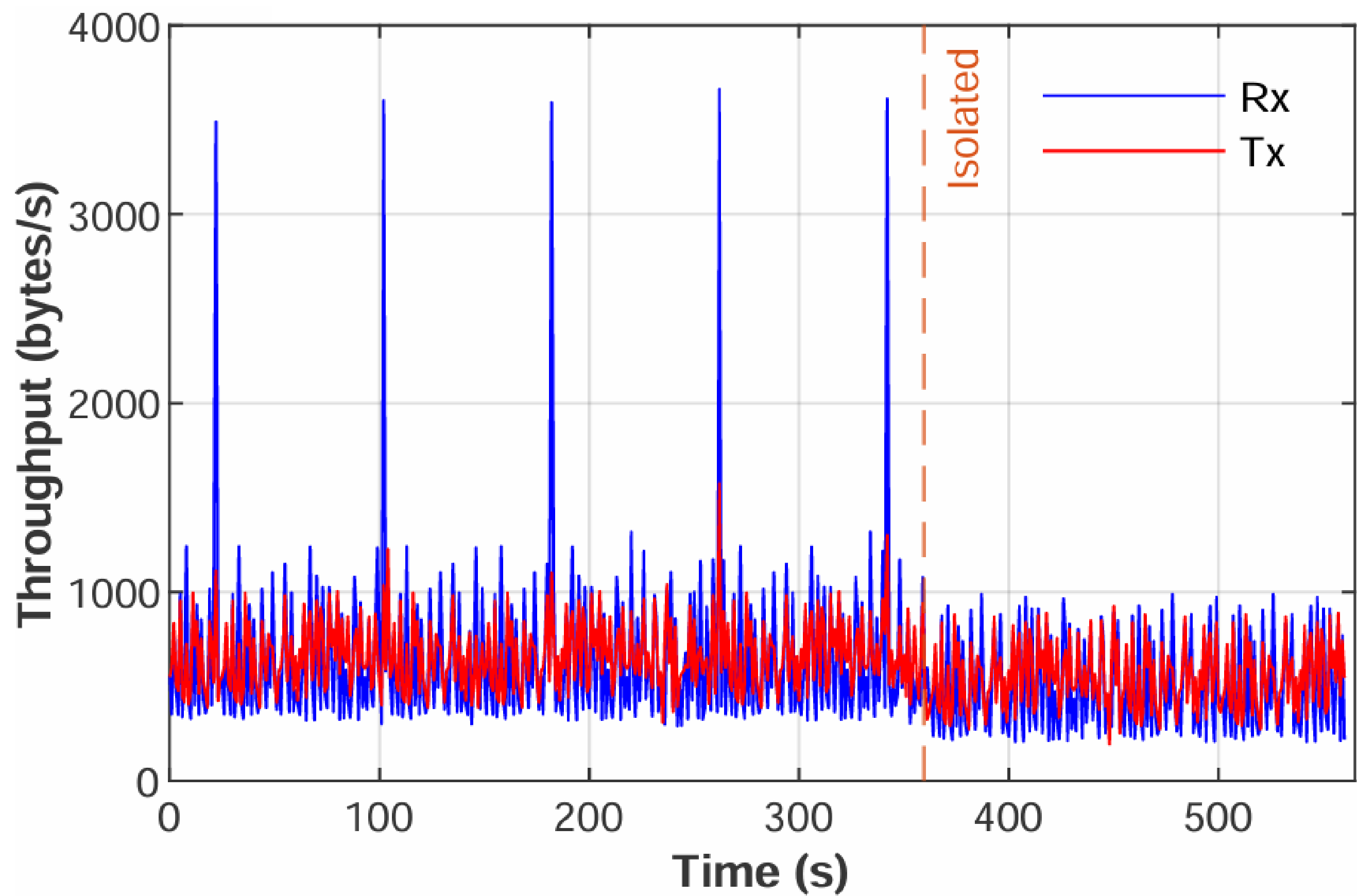


Figure 11: xApp E2 connectivity and RIC indication (RIC indication peaks stop after isolation)

Scalability: Three Bottlenecks Tested

Stress-tested the detector compute path, policy decision plane and concurrent multi-lock enforcement.

1

Detector compute
80 contexts, p99 = 12 ms

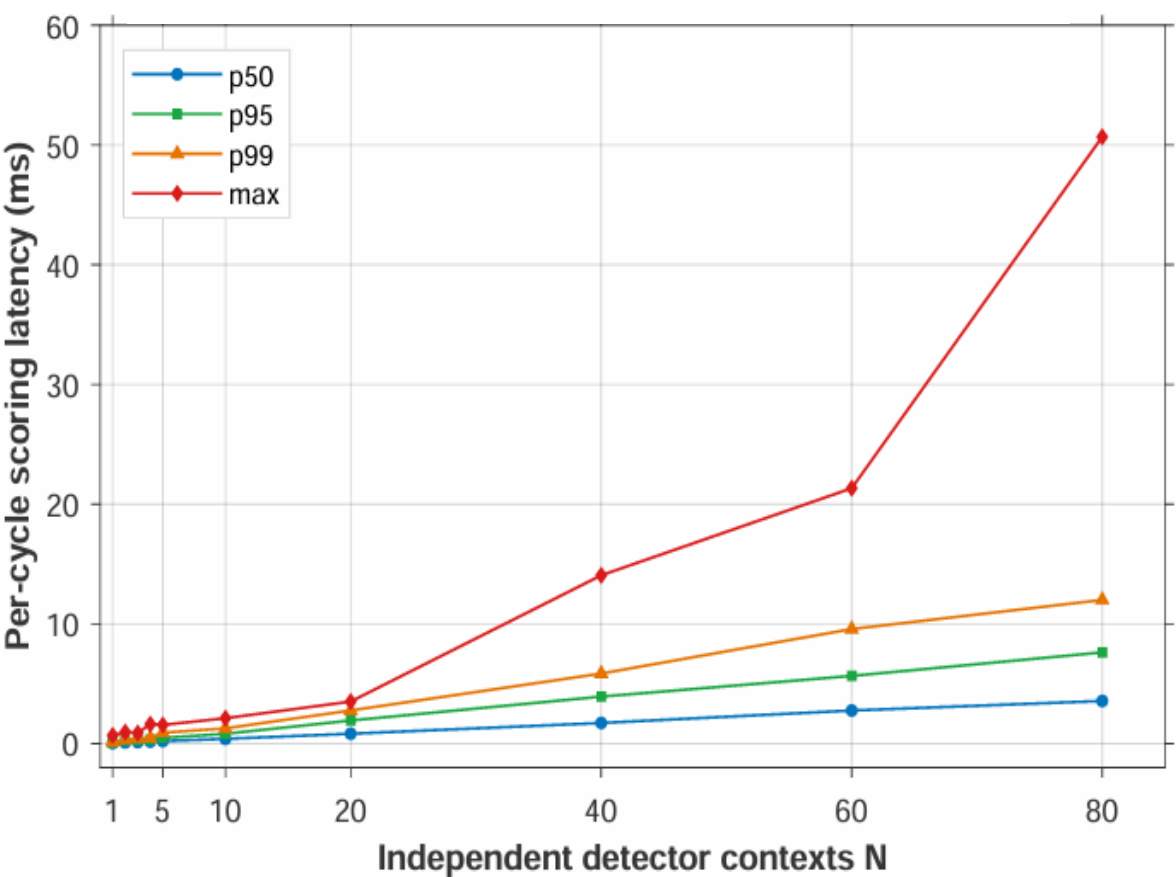


Figure 12: Detector per-cycle scoring latency against the number of independent detector contexts

2

Policy decision plane
Safe through 20 eval/s

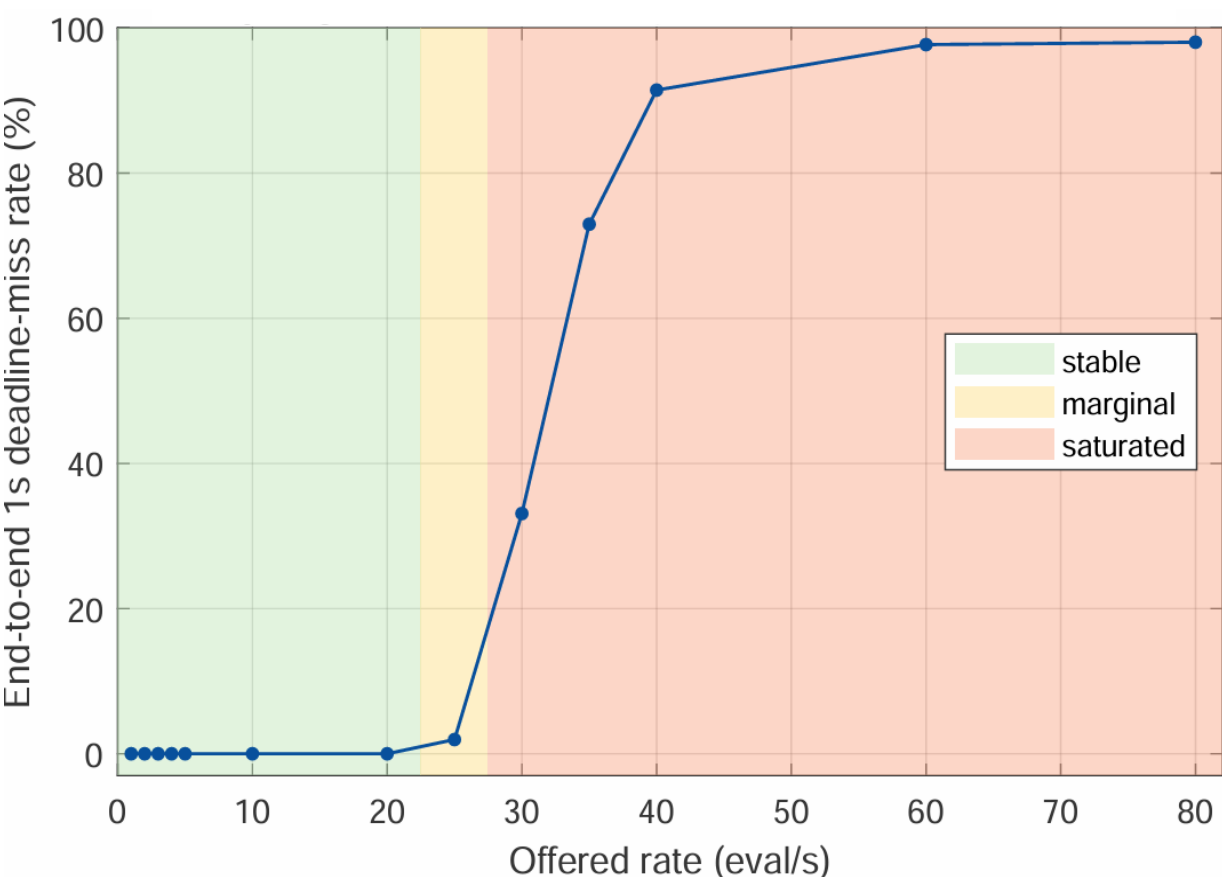


Figure 13: Policy engine c8current decisions 1Hz (saturation knee 25-30)

3

Concurrent containment
K = 1 → 4 compromised xApps

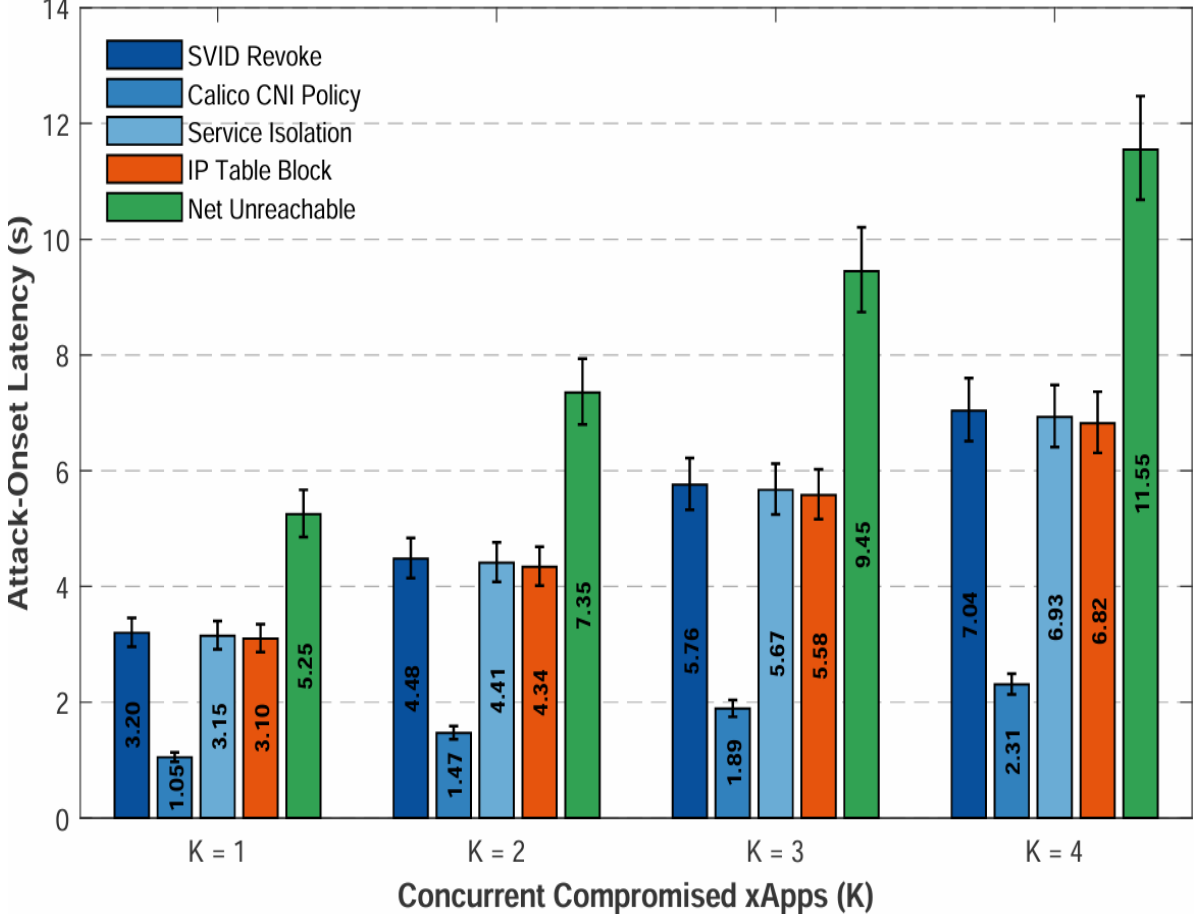


Figure 14: Per stage containment latency vs concurrency



05 Discussion and Conclusion

Challenges and Solutions

Table 03: Issues Encountered and Solutions

Challenge	Solution
Getting live KPM telemetry through E2: kpimon crashed and failed to receive RIC indications	Diagnosed the crash and fixed the telemetry path. The decoding issue was reported.
Distinguishing attacks from legitimate RIC traffic	Created a capability profile for each xApp

Project Goals and Achievements

- **kpimon-go xApp** was patched with the current version of the **ORAN-SC RIC** enabling live KPM/RIC indication.
Source: NIST ORAN Testbed Automation - <https://github.com/usnistgov/O-RAN-Testbed-Automation/issues/13>
- **ZT-XGuard**: A zero-trust framework to defend against **MITRE ATT&CK** influenced adversary techniques.
- **Research outcome**: Conference paper submission to **IEEE Consumer Communications & Networking Conference (CCNC) 2027** presenting the main framework and experimental findings (under review).

Conclusion

- This framework provides lifecycle-spanning protection for O-RAN xApps via **Secure Onboarding**
 - **Dual-channel** monitoring improves coverage by combining behavioral and resource-abuse threat detection
 - A three-state trust model converts evidence into action: NORMAL, SUSPICIOUS and COMPROMISED. (**12ms** Computational Latency(p99), **if 80 concurrent detectors** are worked. This is acceptable for O-RAN operations)
 - Automated containment is enforced using four different containment mechanisms (**4-5.5 s** containment is **acceptable** latency compared with **5/5/5 s**)
 - Containment is verified, rather than relying on an API call
- “ZT-XGuard demonstrates that trust in an xApp should be continuously reassessed throughout its lifecycle, with verified containment when that trust fails”**

Future Work

- Validate and generalize the statistical detector across multiple xApps.
- Extend detection to application-level RAN behavior, including attacks such as E2 data poisoning and malicious but well-formed control logic.

References

- [1] O-RAN Alliance, O-RAN Architecture Description, ORAN.WG1.TS.OAD-R004-v15.00, 2025.
- [2] M. Polese et al., “Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges,” IEEE Communications Surveys & Tutorials, vol. 25, no. 2, pp. 1376–1411, 2023.
- [3] M. Liyanage et al., “Open RAN security: Challenges and opportunities,” Journal of Network and Computer Applications, vol. 214, 2023.
- [4] O-RAN Alliance, Study on Security for Near-RT RIC and xApps, ORAN.WG11.TR.Security-Near-RT-RIC-xApps.0-R004-v06.00, 2025.
- [5] O-RAN Alliance, Study on Continuous Security Monitoring, ORAN.WG11.TR.CSM-Security-R005-v02.00, 2026.
- [6] C.-F. Hung et al., “Security threats to xApps access control and E2 interface in O-RAN,” IEEE Open Journal of the Communications Society, vol. 5, pp. 1197–1203, 2024.
- [7] S. Rose et al., Zero Trust Architecture, NIST SP 800-207, 2020.
- [8] T. O. Atalay et al., “Securing 5G OpenRAN with a scalable authorization framework for xApps,” IEEE INFOCOM, 2023.
- [9] P. Fernando et al., “Securing xApps in open RAN: A hierarchical approach to authentication and authorisation,” IEEE CNS, 2025.
- [10] C.-F. Hung, C.-H. Tseng, and S.-M. Cheng, “Anomaly detection for mitigating xApp and E2 interface threats in O-RAN near-RT RIC,” IEEE Open Journal of the Communications Society, 2025.

References

- [11] A. S. Abdalla et al., “ZTRAN: Prototyping zero trust security xApps for open radio access network deployments,” IEEE Network, 2024.
- [12] D. Lin et al., “ZT-RIC: A zero trust RIC framework for ensuring data privacy and confidentiality in open RAN,” IEEE CCNC, 2025.
- [13] The Falco Project (CNCF), Falco: Cloud-Native Runtime Security, 2026.
- [14] SPIFFE Project (CNCF), The SPIFFE/SPIRE Workload Identity Framework, 2026.
- [15] O-RAN Alliance, Near-RT RIC Architecture, ORAN.WG3.TS.RICARCH-R004-v07.00, 2025.
- [16] Sigstore Project, Cosign: Container Signing, Verification and Storage in an OCI Registry, 2026.
- [17] S. Torres-Arias et al., “in-toto: Providing farm-to-table guarantees for bits and bytes,” 28th USENIX Security Symposium, pp. 1393–1410, 2019.
- [18] OpenSSF, SLSA: Supply-chain Levels for Software Artifacts, 2026.
- [19] The Helm Authors, Helm: The Package Manager for Kubernetes, 2026.
- [20] M. A. Vieira et al., “Fast packet processing with eBPF and XDP: Concepts, code, challenges, and applications,” ACM Computing Surveys, vol. 53, no. 1, 2020.
- [21] P. J. Rousseeuw and C. Croux, “Alternatives to the median absolute deviation,” Journal of the American Statistical Association, vol. 88, no. 424, pp. 1273–1283, 1993.

References

- [22] C. A. Lowry et al., “A multivariate exponentially weighted moving average control chart,” *Technometrics*, vol. 34, no. 1, pp. 46–53, 1992.
- [23] O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004.
- [24] H. Hotelling, “The generalization of Student’s ratio,” *Annals of Mathematical Statistics*, vol. 2, no. 3, pp. 360–378, 1931.
- [25] S. S. Wilks, “Determination of sample sizes for setting tolerance limits,” *Annals of Mathematical Statistics*, vol. 12, no. 1, pp. 91–96, 1941.
- [26] Tigera, Project Calico: Cloud-Native Networking and Network Policy, 2026.
- [27] NIST, NIST 5G Open-Source Testbed Automation Tool, 2026.
- [28] Open5GS Project, Open5GS: Open-Source 5G Core and EPC, 2026.
- [29] Software Radio Systems, srsRAN Project: Open-Source 5G RAN, 2026.
- [30] C. I. King, stress-ng: A Tool to Load and Stress a Computer System, 2026.
- [31] InGuardians, Peirates: Kubernetes Penetration Testing Tool, 2026.
- [32] A. Sultana et al., “Experimental evaluation of xApp conflict mitigation framework in O-RAN: Insights from testbed deployment in OTIC,” in *Proc. IEEE INFOCOM Workshops (INFOCOM WKSHPS)*, 2025.

Thank You